Laboratory 6
Object detection

Objectives

In this laboratory, you will learn about object detection, the task of localizing and classifying **multiple** objects within an image. Unlike semantic segmentation, which assigns a label to every pixel, object detection predicts a set of objects, each described by a class label and a bounding box.
The format of this laboratory will be a bit different. You don't need to write the model code from scratch or get your hands dirty by modifying an existing architecture. Instead you will use two popular object detection models and compare them. The laboratory is designed to mimic a real project: you will have some (messy) data, you need to process it and understand it, read about the two architectures that you can use, and then compare them. Also, you will experiment with generating synthetic data.

Proposed problems

Evaluating Object Detectors

You'll start by understanding the metrics used in evaluating object detectors. Unlike classification or semantic segmentation, object detection must determine both **what** was detected and **where** it was detected. A prediction is only correct if the model predicts the right class and localizes the object accurately.
For this reason, object detection metrics are built on two ideas: overlap between bounding boxes and matching predicted objects to ground-truth objects.

Intersection over Union
Intersection over Union (IoU) measures how well a predicted bounding box overlaps with a ground-truth bounding box. It is defined as the ratio between the area of overlap and the area of union of the two boxes. IoU takes values in the range [0, 1]. A value of 1 indicates perfect alignment, while a value of 0 indicates no overlap. In object detection, IoU is not a metric by itself, but a building block: it is used to decide whether a prediction is considered a **true positive**.

A prediction is considered correct if its IoU with a ground-truth box exceeds a fixed threshold (for example 0.7).

Precision and Recall
Once predictions are matched to ground-truth boxes using an IoU threshold, we can define precision and recall. You are already familiar with these two from image classification.
Precision measures how many predicted boxes are actually correct. It answers the question: *when the model predicts an object, how often is it right?*
Recall measures how many ground-truth objects were detected. It answers the question*: how many of the real objects did the model find?*

There is a trade-off between precision and recall. A model that predicts many boxes may achieve high recall but low precision, while a conservative model may have high precision but miss many objects.

Precision-Recall curve

Object detectors output a confidence score for each predicted bounding box. By varying the confidence threshold used to accept predictions, we obtain different precision-recall pairs.

Plotting precision as a function of recall gives the precision-recall (PR) curve. This curve shows how detection quality changes as we move from conservative to aggressive prediction behavior.

PR curves are more informative than a single precision or recall value, especially in cases of imbalanced datasets.

Average Precision (AP)

Average Precision (AP) summarizes the precision-recall curve into a single number. It is defined as the area under the precision-recall curve. AP is computed per class, using a fixed IoU threshold (for example AP@0.5). A higher AP indicates better detection performance for that class.

Average Precision is computed **per class**. The inputs are the predicted boxes for that class (with confidence scores) and the ground-truth boxes for that class. First sort the predictions by confidence and then, going from highest to lowest confidence, decide whether each prediction is a true positive or a false positive based on an IoU threshold.

Algorithm for AP for one class c at IoU threshold τ is:

1. Collect predictions and ground truth
2. Sort prediction list by confidence score in **descending** order.
3. For each prediction in sorted order:
    a. Analyze only at the ground-truth boxes from the same image that belong to class c
    b. Compute IoU between the prediction and every **unmatched** ground-truth box (**!!! one ground-truth can be matched at most once !!!**).
    c. Find the ground-truth box with the highest IoU.
        i. If the best IoU $\geq\tau$ and that ground-truth box has not been matched yet
        Mark the prediction as TP and mark the ground-truth as "matched".
        ii. Otherwise mark the prediction as FP.
    d. Build precision and recall arrays: Let TP_k be the number of true positives among the first k prediction in sorter order, and FP_k the number of false positives among the first k. Also, let N_{gt} be the total number of ground-truth boxes of class c.
    For each $k = 1, 2, \ldots, K$
    $$Precision(k) = \frac{TP_k}{TP_k + FP_k} \text{ and } Recall(k) = \frac{TP_k}{N_{gt}}$$
    This gives you a discrete precision-recall curve.
    e. Compute AP (area under the PR curve):
    $$AP = \sum_{k=1}^{K} (Recall(k) - Recall(k-1))Precision(k)$$
    , where K is the total number of predicted boxes for a given class.

Finally, apply "precision envelope" correction (to obtain a monotonic precision), which makes AP stable: traverse precision values from the end to the start and replace each precision by the maximum precision observed for any higher recall. This ensures precision does not increase as recall increases, which is a standard convention in detection evaluation.

For k = K-1, .., 2, 1:

$$Precision(k) = max(Precision(k), Precision(k+1))$$

mean Average Precision (mAP)

Once you have the AP for one class, mAP is the average AP across all classes.
If you evaluate at a single IoU threshold τ:

$$mAP_{\tau} = \frac{1}{C}\sum_{i=1}^{C} AP_c(\tau),$$

where C is the number of classes.

When implementing these metrics from scratch, the goal is not to produce a fast or reusable evaluation library (although you could also do this :)). In real projects, you would never do this and you would rely on standard, well-tested implementations. The purpose of this exercise is to make you **think** about how object detection is evaluated and what each metric actually captures.

- Mean Average Precision (mAP) provides an aggregated view of detector performance. It captures both localization quality and confidence calibration. It is the standard metric used to compare object detectors, but its compactness can hide specific failure cases.
- Intersection over Union (IoU) focuses on localization accuracy. It is used when the exact spatial extent of objects matters, such as in medical imaging or autonomous driving. IoU makes it explicit that a correct class prediction is not sufficient if the object is poorly localized.
- Precision measures how reliable the model's predictions are. High precision means that when the detector fires, it is usually correct. This is important in applications where false positives are costly or dangerous.
- Recall measures how complete the detections are. High recall indicates that the model finds most of the objects present in the image. This is important in scenarios where missing an object is more problematic than producing extra detections.
- The F1 score combines precision and recall into a single value, making it useful when there is a need to balance false positives and false negatives. This is important when comparing models that trade precision for recall in different ways.

As with semantic segmentation, object detection is a strongly visual task. Two models with similar mAP values can behave very differently. One may produce many duplicate boxes, another may miss small objects, and another may mislocalize boundaries.

Complement quantitative metrics with visual inspection of predicted bounding boxes. Logging predictions and ground truth side by side in *wandb*.

Finally, you will be given **three case studies**, each corresponding to a "trained" object detector evaluated on the same dataset. For each case study, you will be given the predicted bounding boxes and the ground truth bounding boxes in yolo format.

Your task is to analyze and compute the metrics for each case and reason about what they reveal about the model's behavior. You should describe *why* the metrics take the observed values and what kinds of failure modes they suggest. Go beyond restating the numbers: visualize the predictions and think about how the detector is behaving in terms of localization, confidence, and class-wise performance.

For each case study, also propose concrete actions you could take in a real project to improve the model. This may include changes to confidence thresholds, training strategy, loss design, data balance, augmentation, or model architecture. The goal here is to practice interpreting metrics as diagnostic tools rather than as isolated scores.

Data and Data Processing

Your task for this laboratory will be to train and compare to object detectors on the task of flower detection. As mentioned before, one of the objectives is to learn how to handle un-curated data. So, for this laboratory you won't be given a clean dataset with object labels in yolo format, instead you have to build the dataset from various sources (that have different annotation formats or even no annotations for object detection).
You can choose any problem you want, the flower dataset is just an example.
You should combine at least **two** data sources (the more the better). For starting you can check out the following datasets:

- https://universe.roboflow.com/orbit-final-project/flower-detection-fscsr
- https://www.robots.ox.ac.uk/~vgg/data/flowers/102/index.html
- https://proai-datasets.s3.eu-west-3.amazonaws.com/progetto-finale-flowes.tar.gz

After gathering the data, think on how you should define the train, validation and test split.
Write a **short** dataset card in which you describe all the details of the dataset you will use (number of samples, number of classes, dataset splits, annotation format etc.). This is not just an academic exercise. In real projects, undocumented datasets really create confusion and reproducibility issues.

Finally, visualize your dataset. Do at least the following:

- create a chart showing the number of bounding boxes per class. This will immediately show class imbalance.
- analyze object sizes. Compute bounding box width, height, and area (both absolute and relative to image size) and visualize their distributions using histograms. This allows you to understand whether most flowers are small, medium, or large within the image.

Understanding these statistics early will make your later results much easier to interpret, as object detection performance is strongly linked to data distribution.

Generating synthetic data

One experiment that you will perform in this laboratory is to generate synthetic data and evaluate whether adding it to your training set improves detection performance. Synthetic data here means new images with

automatically generated annotations, not just simple augmentations of existing ones. You can try **any** of the following or propose a completely new pipeline:

- Copy-Paste Composition: Extract flower instances (using their bounding boxes or segmentation masks if available or use SAM) and paste them onto different background images to create new scenes. Vary placement, scale, and orientation to make these composites realistic. This does not require expensive modeling or generative training.
- Diffusion inpainting for background replacement. Keep the flower region fixed (mask it), and use inpainting to generate different plausible backgrounds around it: different vegetation, lighting feel, clutter, seasonal colors, etc.

Object detection models

Now that you understand the evaluation metrics and you cleaned and processed the data, it is time to train object detection models. In this laboratory, you will work with two state-of-the-art detectors (as of 2025): YOLO26 and RF-DETR. During the lectures, we discussed the YOLO family of models and the DETR line of object detectors, but not these specific architectures

Before training anything, your first task is to study the original papers of YOLO26 and RF-DETR and understand their architectures at a high level. You can consult additional resources such as blog posts or explanatory videos, but you should not skip reading the papers themselves. Do not memorize implementation details, but try to understand the design principles behind each model.

Analyze the experimental results reported in the papers. Identify the scenarios in which each model excels and the cases where it struggles, such as small objects, crowded scenes, or computational constraints.
Finally, compare the two models in terms of computational complexity. Consider training cost, inference latency, and hardware requirements. This analysis will help you reason about which model is more suitable for different real-world applications.

For training you will use the repo/pip packages of these models, you don't have to write any code.

To sum up, your tasks for this lab are:

- Implement object detection metrics from scratch (IoU, Precision, Recall, AP, mAP) and analyze the three provided case studies.
- Build and document your **own flower detection dataset** by combining multiple sources, cleaning them, defining splits, and visualizing class and object size distributions.
- Design one synthetic data pipeline (and evaluate whether adding synthetic samples improves detection performance - see next TODO).
- Read and analyze the YOLO26 and RF-DETR papers, focusing on architecture principles, strengths, weaknesses, and computational complexity.
- Train both models, track experiments with *wandb*, evaluate them properly, and compare real-only vs real+synthetic performance.